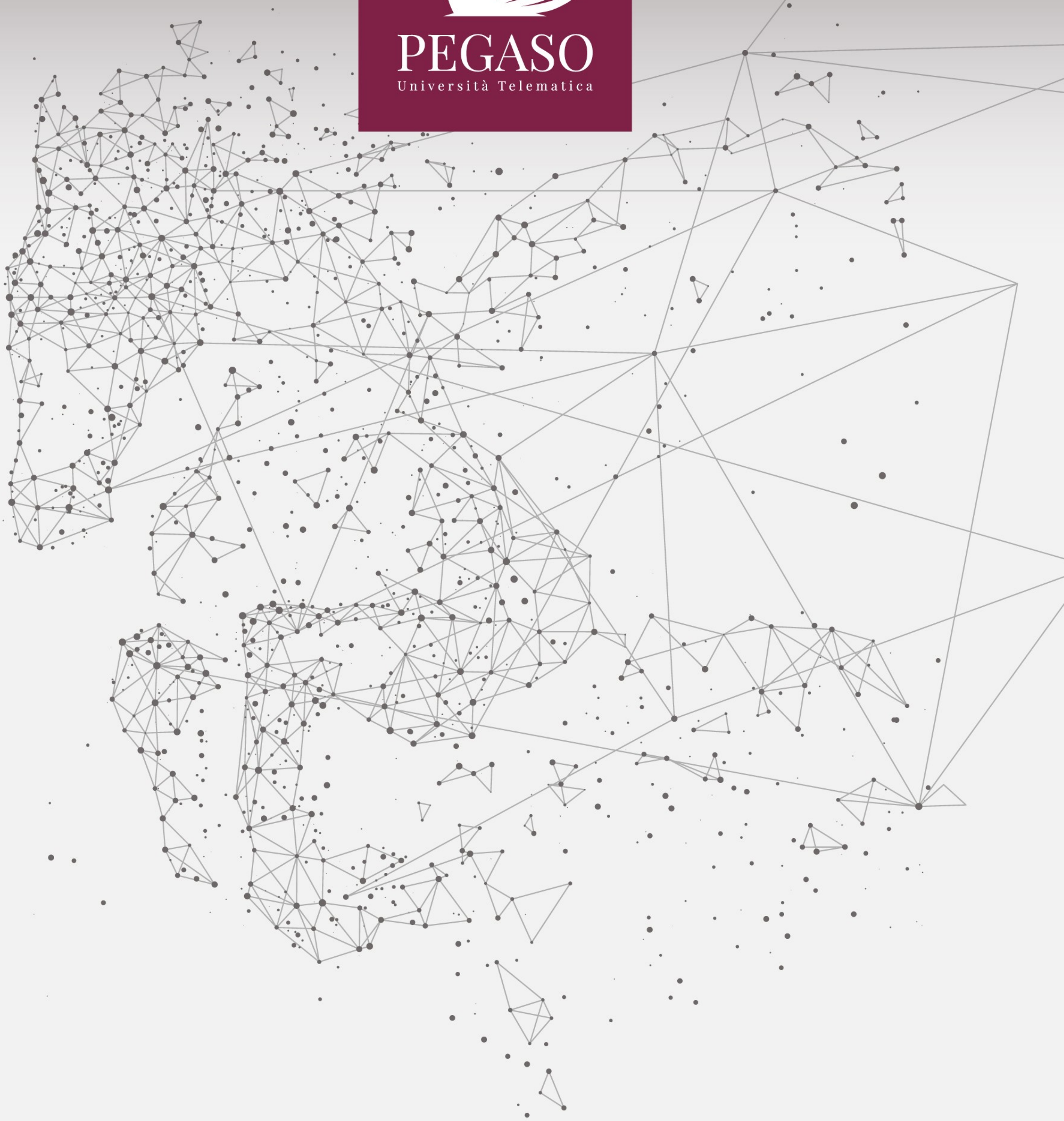
PEGASO
Università Telematica



Indice

1. INTRODUZIONE 3
2. DA NUMERO A CENTO CON PASSO 2 4
3. DA NUMERO A CENTO CON PASSO N 6
4. SOMMA NUMERI RELATIVI 8
5. SEQUENZA CRESCENTE 10

BIBLIOGRAFIA 14

1. Introduzione

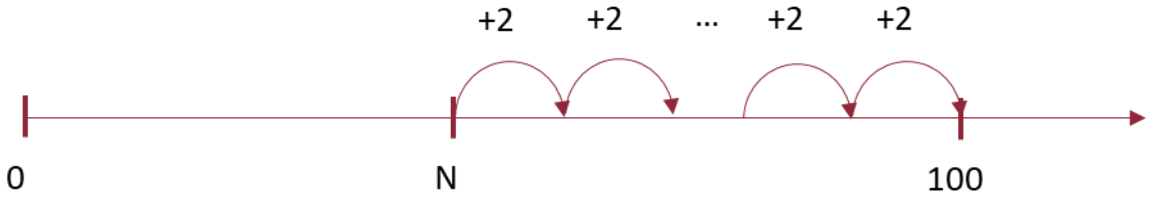
In questa dispensa di accompagnamento alla video-lezione, si illustrano ed analizzano alcune esercitazioni molto importanti che approfondiscono i concetti e gli esercizi svolti finora. In particolare, si riportano quattro esercizi sulle strutture iterative. I primi due esercizi permettono di valutare la necessità dell’utilizzo di strutture iterative nel caso in cui non si conosca il numero di iterazioni da eseguire. Il terzo esercizio riguarda l’uso di un’unica struttura iterativa per gestire due flussi diversi di elaborazione. Il quarto esercizio mostra l’utilizzo combinato di strutture iterative, strutture di selezione e flag.

2. Da numero a cento con passo 2

L'esercizio proposto è quello di chiedere un numero in input e calcolare quante volte bisogna sommare due a tale numero per arrivare/superare 100. L'esercizio, ovviamente, si può risolvere in modo molto semplice matematicamente, ma l'obiettivo dell'esercizio è quello di esercitarsi con le strutture iterative per cui non si conosce a priori il numero di iterazioni che dovranno essere effettuate.

Ad esempio, inserendo il numero 50, il programma dovrà aggiungere una quantità pari a 2 finché non si raggiunge/supera il valore 100 (nell'esempio 25 volte).

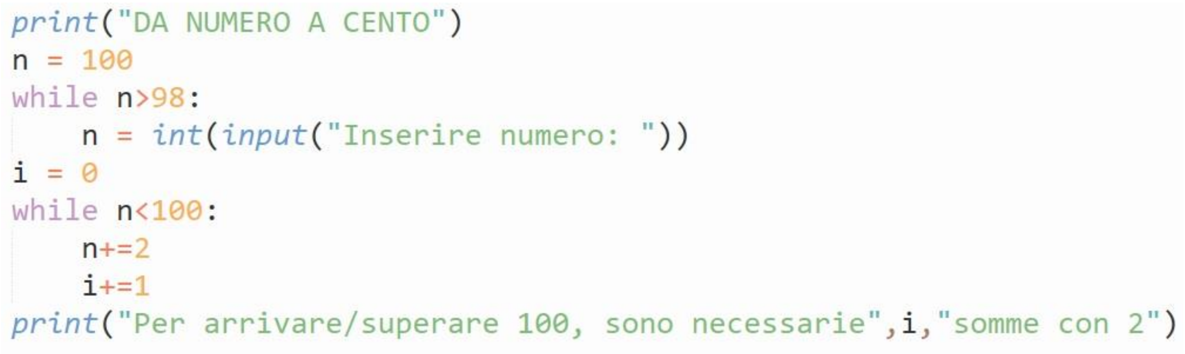
Nella figura seguente si propone uno schema esemplificativo del problema da risolvere.



Il codice Python dell'esercizio è riportato di seguito. Viene inizializzata la variabile n a 100 in modo tale da entrare nel ciclo di validazione dell'input. Vincolo di tale ciclo è che il numero da inserire sia minore o uguale a 98. Viene, inoltre, inizializzata la variabile i che conterà il numero delle somme con 2 del numero inserito prima in input.

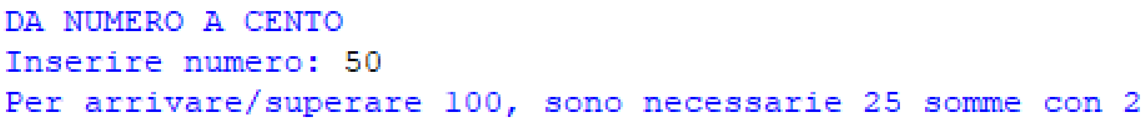
Successivamente, nel codice viene inserita una struttura iterativa (while) che itera finché la condizione n<100 è vera. Nel corpo del ciclo, ad ogni iterazione, si procede a sommare 2 al numero che man mano si incrementa e si incrementa di una unità il valore della variabile per calcolare il numero delle somme.

Al termine dell'elaborazione, a schermo verranno visualizzati il numero inserito in input e il numero di somme con 2 necessarie.



```
print("DA NUMERO A CENTO")
n = 100
while n>98:
    n = int(input("Inserire numero: "))
i = 0
while n<100:
    n+=2
    i+=1
print("Per arrivare/superare 100, sono necessarie",i,"somme con 2")
```

Di seguito si riporta il risultato dell’elaborazione del codice precedente per un numero iniziale inserito in input pari a 50.



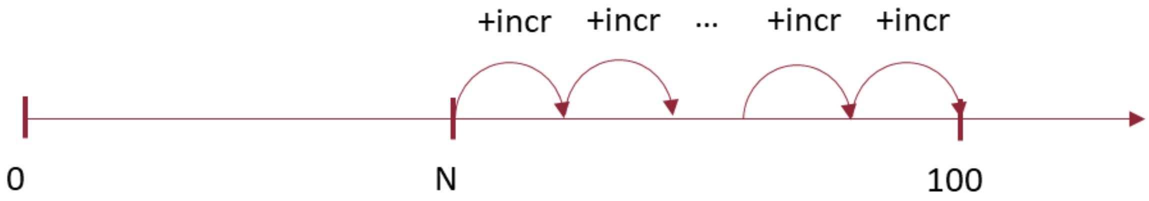
```
DA NUMERO A CENTO
Inserire numero: 50
Per arrivare/superare 100, sono necessarie 25 somme con 2
```

3. Da numero a cento con passo n

L’esercizio proposto è quello di chiedere due numeri in input all’utente (N e incr) e calcolare quante volte bisogna sommare incr a N per arrivare/superare 100. L’esercizio, ovviamente, si può risolvere in modo molto semplice matematicamente, ma l’obiettivo dell’esercizio è quello di esercitarsi con le strutture iterative per cui non si conosce a priori il numero di iterazioni che dovranno essere effettuate.

Ad esempio, inserendo N = 20 e incr=10, il programma dovrà aggiungere incr a N finché non si raggiunge/supera il valore 100 (nell’esempio 8 volte).

Nella figura seguente si propone uno schema esemplificativo del problema da risolvere.



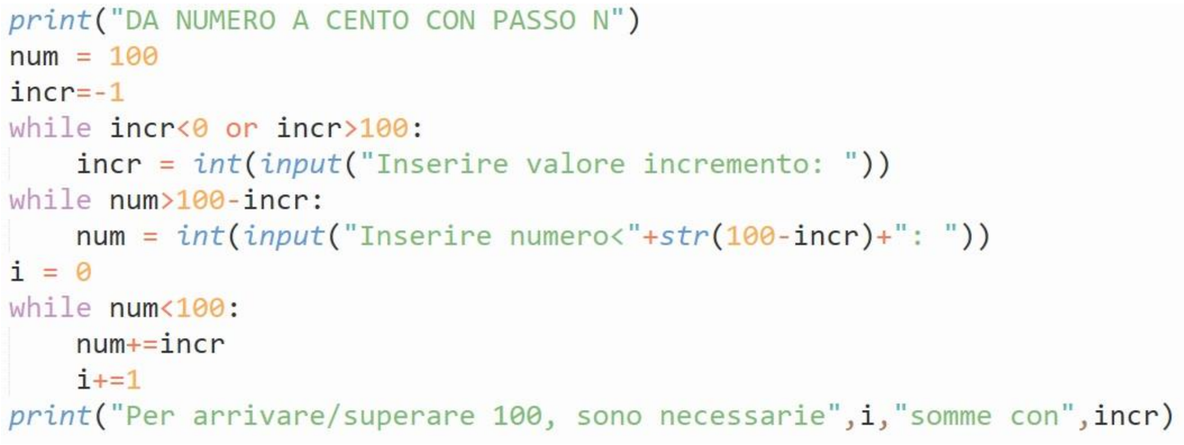
Il codice Python dell’esercizio è riportato di seguito. Viene inizializzata la variabile num a 100 e la variabile incr a -1 per poter “entrare” nei due cicli di validazione dell’input.

Vincolo del primo ciclo di validazione dell’input è che l’incremento incr da inserire sia positivo e minore o uguale a 100. Vincolo del secondo ciclo di validazione dell’input è che il numero da cui partire per le somme sia un numero minore o uguale alla differenza tra 100 e il valore inserito precedentemente di incr.

Viene, inoltre, inizializzata la variabile i che conterà il numero delle somme di num con incr.

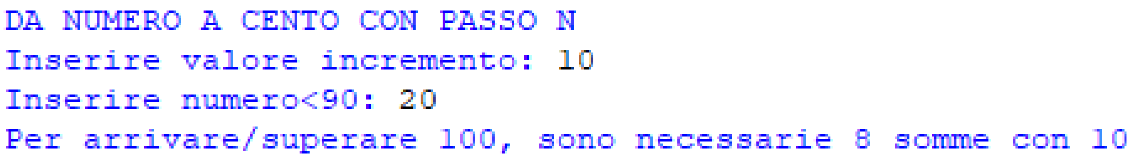
Successivamente, nel codice viene inserita una struttura iterativa (while) che itera finché la condizione n<100 è vera. Nel corpo del ciclo, ad ogni iterazione, si procede a sommare incr al numero num che man mano si incrementa e si incrementa di una unità il valore della variabile per calcolare il numero delle somme effettuate.

Al termine dell’elaborazione, a schermo verranno visualizzati l’incremento inserito in input e il numero di somme tra num e incr necessarie.



```python
print("DA NUMERO A CENTO CON PASSO N")
num = 100
incr=-1
while incr<0 or incr>100:
    incr = int(input("Inserire valore incremento: "))
while num>100-incr:
    num = int(input("Inserire numero<"+str(100-incr)+": "))
i = 0
while num<100:
    num+=incr
    i+=1
print("Per arrivare/superare 100, sono necessarie",i,"somme con",incr)
```

Di seguito si riporta il risultato dell'elaborazione del codice precedente per un incremento di 10 e numero iniziale di 20.



```
DA NUMERO A CENTO CON PASSO N
Inserire valore incremento: 10
Inserire numero<90: 20
Per arrivare/superare 100, sono necessarie 8 somme con 10
```

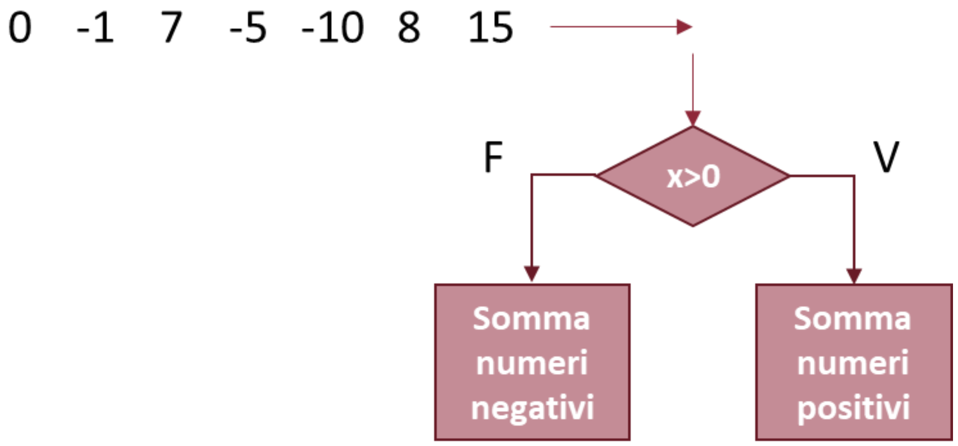
4. Somma numeri relativi

L'esercizio proposto è il seguente:

- data in input una serie di N numeri positivi e negativi, calcolare due somme distinte: la somma dei numeri positivi e la somma dei numeri negativi;
- al termine dell'inserimento della sequenza, calcolare i valori assoluti delle due somme;
- stabilire quale delle due somme in valore assoluto è la maggiore.

Obiettivo dell'esercizio è quello di utilizzare un'unica struttura iterativa per gestire due flussi diversi di elaborazione.

Nella figura seguente si propone uno schema esemplificativo del problema da risolvere. In particolare, una sequenza di numeri relativi che terminerà con il valore zero, viene acquisita in input. Per ciascuno dei numeri inseriti in input, si verifica se tale numero è positivo o negativo. Il numero della sequenza viene quindi sommato all'"accumulatore" dei numeri positivi o negativi, rispettivamente.



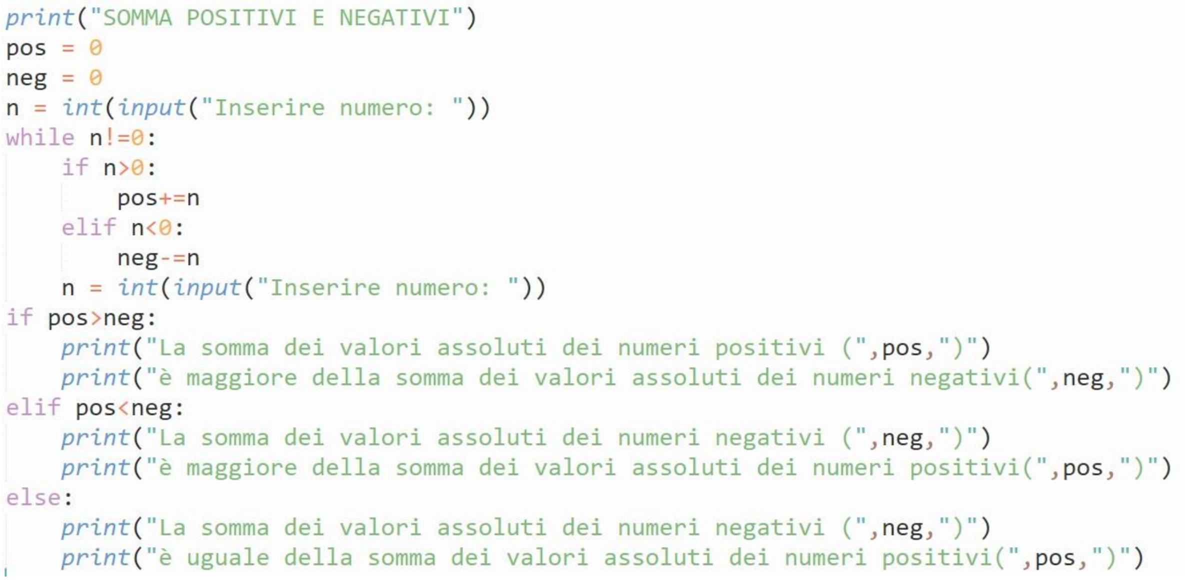
Il codice che segue implementa l'algoritmo richiesto. Innanzitutto vengono inizializzati a zero l'accumulatore dei numeri negativi (neg) e l'accumulatore dei numeri positivi (pos).

Viene richiesto il primo numero n e, successivamente, si comincia il ciclo di inserimento della sequenza numerica di input. Per ciascun numero si verifica se esso è positivo o negativo (se fosse zero si interromperebbe l'inserimento della sequenza numerica).

Se il numero inserito all'i-esima iterazione è positivo, si somma all'accumulatore dei numeri positivi (pos), altrimenti si somma all'accumulatore dei numeri negativi (neg).

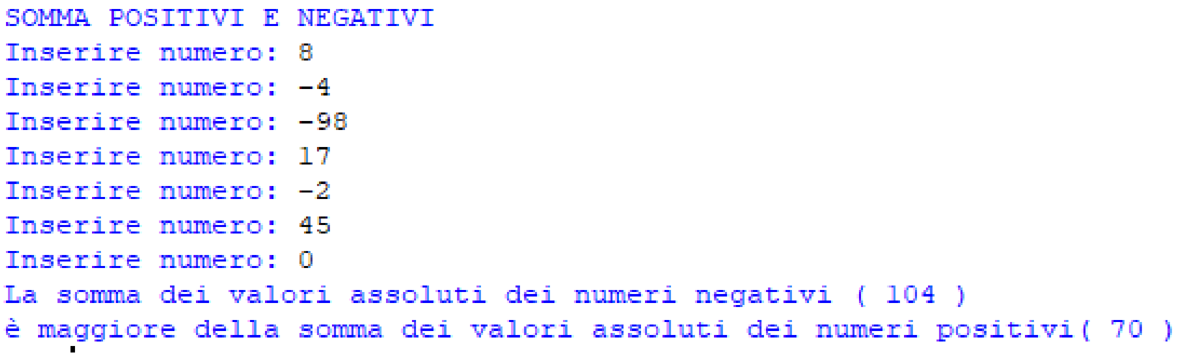
Per evitare il calcolo dei valori assoluti al termine della sequenza di input ed elaborazione, i numeri negativi vengono invertiti di segno prima di essere “accumulati” in neg.

Al termine della sequenza di input, verrà visualizzato uno dei tre messaggi previsti per le tre casistiche possibili: pos >neg, pos < neg, pos = neg.



```
print("SOMMA POSITIVI E NEGATIVI")
pos = 0
neg = 0
n = int(input("Inserire numero: "))
while n!=0:
    if n>0:
        pos+=n
    elif n<0:
        neg-=n
    n = int(input("Inserire numero: "))
if pos>neg:
    print("La somma dei valori assoluti dei numeri positivi (",pos,")")
    print("è maggiore della somma dei valori assoluti dei numeri negativi(",neg,")")
elif pos<neg:
    print("La somma dei valori assoluti dei numeri negativi (",neg,")")
    print("è maggiore della somma dei valori assoluti dei numeri positivi(",pos,")")
else:
    print("La somma dei valori assoluti dei numeri negativi (",neg,")")
    print("è uguale della somma dei valori assoluti dei numeri positivi(",pos,")")
```

Di seguito si riporta il risultato dell’elaborazione del codice precedente per la seguente sequenza numerica: 8, -4, -98, 17, -2, 45, 0.



```
SOMMA POSITIVI E NEGATIVI
Inserire numero: 8
Inserire numero: -4
Inserire numero: -98
Inserire numero: 17
Inserire numero: -2
Inserire numero: 45
Inserire numero: 0
La somma dei valori assoluti dei numeri negativi ( 104 )
è maggiore della somma dei valori assoluti dei numeri positivi( 70 )
```

5. Sequenza crescente

All'interno di una sequenza numerica che viene acquisita in input c'è un'unica sottosequenza crescente.

Obiettivo dell'esercizio è stabilire quanto è lunga la sottosequenza sia in termini numerici (es. 6 numeri), sia in percentuale sul totale dei numeri dell'intera sequenza (es. 70%). L'esercizio proposto dovrà, quindi:

- chiedere in input la lunghezza della sequenza;
- permettere l'inserimento dei numeri della sequenza;
- stabilire quando inizia e quando termina la sottosequenza crescente;
- calcolare quanti numeri fanno parte della sottosequenza positiva e la percentuale degli stessi rispetto all'intera sequenza.

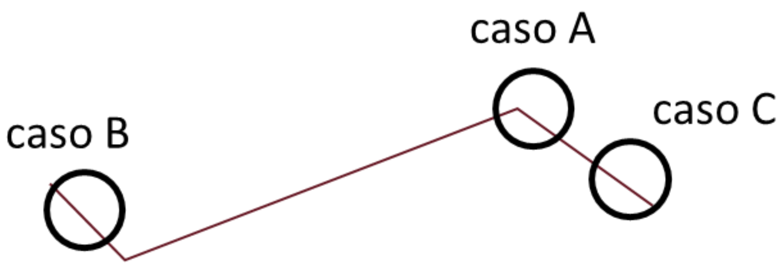
Obiettivo dell'esercizio è quello di utilizzare una struttura iterativa per confrontare tra loro numeri in successione. Per tale motivo saranno necessarie due variabili:

1. la prima che conterrà il numero inserito all'iterazione i;
2. la seconda che conterrà il numero inserito all'iterazione i+1 in modo tale da verificare se il numero inserito all'iterazione i+1 sia maggiore, minore o uguale a quello inserito all'iterazione i.

Nel codice sviluppato tali variabili si chiamano num_prec (numero precedente) e num_suc (numero successivo).

Inoltre, andranno accuratamente considerati i seguenti casi:

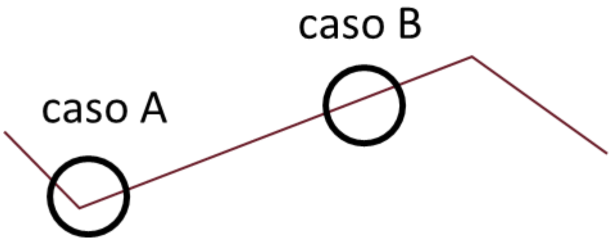
- Il verificarsi della condizione $n_{i+1}<n_i$ (dove n_k è il numero inserito alla k-esima iterazione) può avere due valenze diverse poiché:
    1. potrebbe essere il termine della sottosequenza crescente (caso A riportato nella figura seguente);
    2. ma anche una sequenza decrescente anteriore (caso B riportato nella figura seguente) o posteriore alla sottosequenza crescente (caso C riportato nella figura seguente).



Andranno accuratamente considerati anche i seguenti casi:

- Allo stesso modo il verificarsi della condizione $n_{i+1} \geq n_i$ (dove n_k è il numero inserito alla k-esima iterazione) può avere due valenze diverse poiché:
    1. potrebbe essere l'inizio della sottosequenza crescente (caso A riportato nella figura seguente);
    2. potrebbe essere parte della sequenza crescente (caso B riportato nella figura seguente).

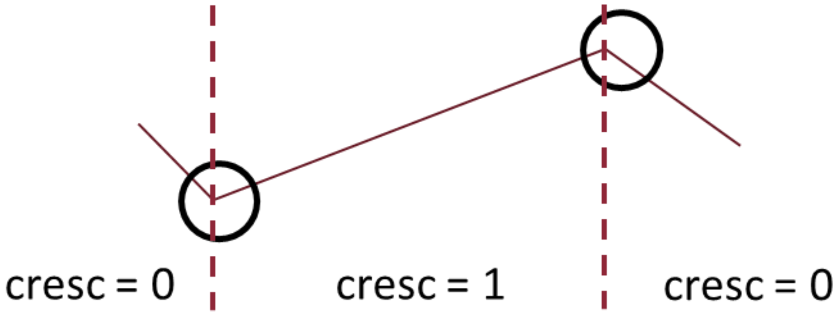
Altri casi oltre a quelli presentati non ci possono essere poiché si ipotizza ci sia una sola sottosequenza crescente nella sequenza di numeri.



Per poter distinguere i casi precedenti tra di loro, sarà necessario utilizzare una variabile flag (bandiera) che ci permette di distinguere quando si considerano numeri al di fuori della sottosequenza crescente e quando si considerano numeri appartenenti alla sottosequenza crescente. In particolare:
il flag cresc sarà inizializzato a zero (cioè sottosequenza crescente «non in corso»);

- alla prima occorrenza della condizione $n_{i+1} \geq n_i$, si procederà a porre il flag a 1 e a contare i numeri della sottosequenza crescente con una variabile contatore apposita;
- per le occorrenze successive della condizione $n_{i+1} \geq n_i$, essendo il flag a 1, si procederà a contare i numeri della sottosequenza crescente;
- alla prima occorrenza della condizione ni+1<ni, si procederà a porre il flag a 0 e a terminare il conteggio della lunghezza della sottosequenza crescente.

La figura seguente schematizza l'andamento del flag per la sequenza di numeri al verificarsi delle condizioni appena enunciate.



Di seguito si riporta il codice che implementa tale algoritmo. Si inizializzano a zero il flag cresc a zero, la lunghezza della sottosequenza crescente l e la lunghezza della sequenza totale N.

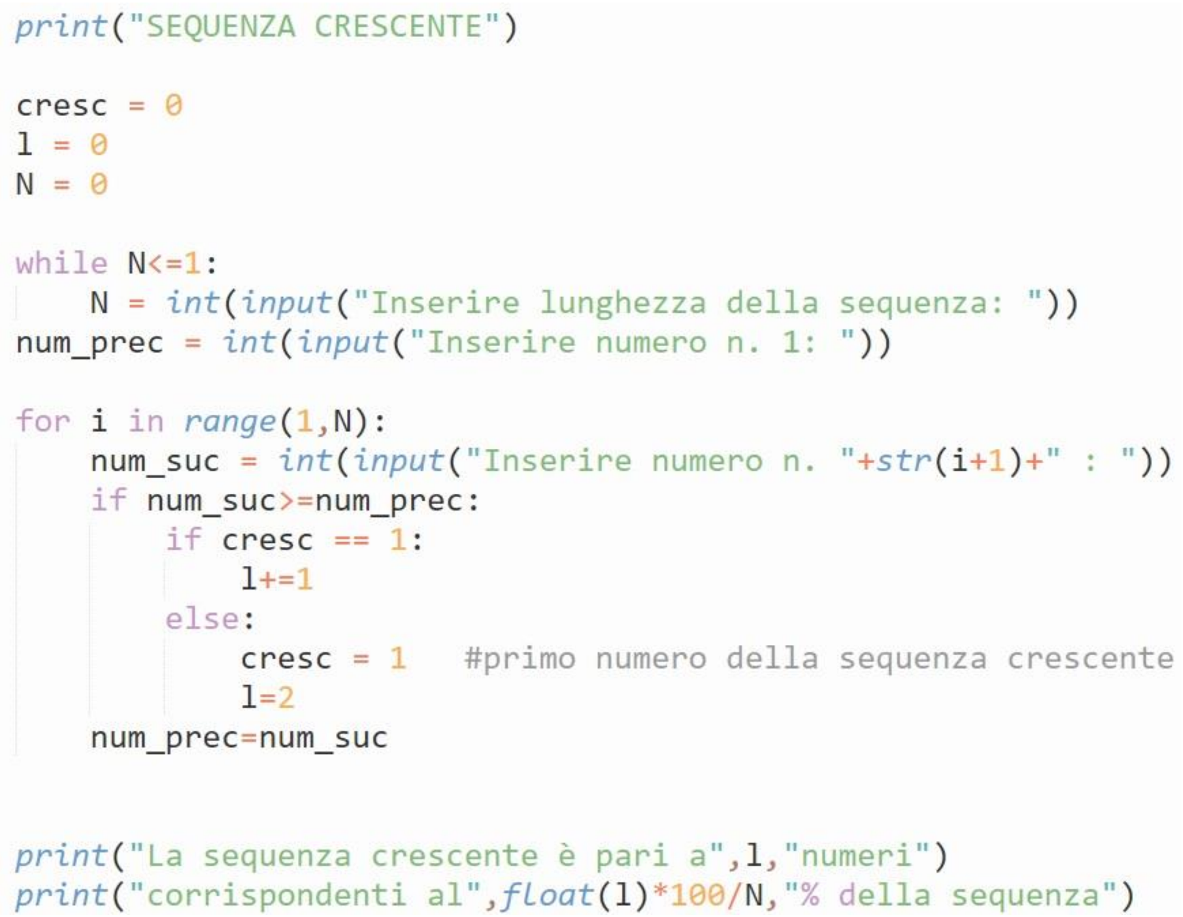
Successivamente con un ciclo di validazione dell'input si impone che N debba essere maggiore di 1.

Quindi, si procede all'inserimento del primo numero della sequenza al di fuori della struttura iterativa proprio perché dovremo andare a confrontare due numeri successivi della sequenza numerica (il primo numero della sequenza verrà salvato nella variabile num_prec). Si imposta la struttura iterativa per ripetere N-1 iterazioni (il primo numero della sequenza è stato già inserito).

Nella struttura iterativa si procede con la lettura del num_suc e con la verifica della condizione num_suc>= num_prec (equivalente a $n_{i+1} \geq n_i$). Se ciò avviene e il flag cresc è a 1 vuol dire che siamo già all'interno della sottosequenza crescente, quindi si andrà solo ad incrementare di 1 la lunghezza della sequenza.

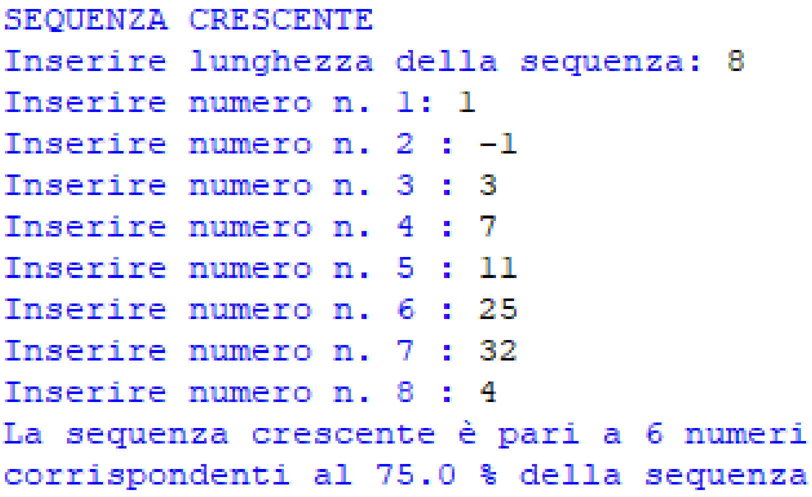
Se la condizione num_suc>= num_prec viene verificata, ma cresc =0, vuol dire che la sottosequenza crescente (unica nella sequenza totale) sta cominciando, quindi il flag viene posto a 1 e la lunghezza della sottosequenza viene inizializzata a 2 poiché due sono i numeri che si sono confrontati.

Al termine del corpo del ciclo for, si procede con l'aggiornamento num_prec= num_suc per continuare il confronto con altri due numeri successivi.



```
print("SEQUENZA CRESCENTE")

cresc = 0
l = 0
N = 0

while N<=1:
    N = int(input("Inserire lunghezza della sequenza: "))
num_prec = int(input("Inserire numero n. 1: "))

for i in range(1,N):
    num_suc = int(input("Inserire numero n. "+str(i+1)+" : "))
    if num_suc>=num_prec:
        if cresc == 1:
            l+=1
        else:
            cresc = 1   #primo numero della sequenza crescente
            l=2
    num_prec=num_suc


print("La sequenza crescente è pari a",l,"numeri")
print("corrispondenti al",float(l)*100/N,"% della sequenza")
```

Di seguito si riporta il risultato dell'elaborazione del codice precedente per la seguente sequenza numerica: 1, -1, 3, 7, 11, 25, 32, 4.



```
SEQUENZA CRESCENTE
Inserire lunghezza della sequenza: 8
Inserire numero n. 1: 1
Inserire numero n. 2 : -1
Inserire numero n. 3 : 3
Inserire numero n. 4 : 7
Inserire numero n. 5 : 11
Inserire numero n. 6 : 25
Inserire numero n. 7 : 32
Inserire numero n. 8 : 4
La sequenza crescente è pari a 6 numeri
corrispondenti al 75.0 % della sequenza
```

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/